

Capítulo 3

Números aleatorios

3.1. Introducción

En simulación estocástica los generadores de números (pseudo)aleatorios con distribución uniforme en el intervalo $[0, 1]$ son empleados para diversos usos:

- en forma directa, es decir, porque se desea obtener valores uniformemente distribuidos en $[0, 1]$,
- para generar muestras de otras variables aleatorias, con distribuciones discretas o continuas,
- para generar muestras de un conjunto de variables aleatorias dependientes, como por ejemplo procesos estocásticos y distribuciones multivariadas.

La ejecución de una simulación está fuertemente correlacionada con el generador de variables aleatorias uniformes utilizado, por lo cual es importante garantizar las buenas propiedades del generador. Dado que los valores de la variable son generados a partir de una secuencia de pasos o algoritmo, sería más preciso referirse a **generadores de números pseudoaleatorios**.

3.2. Propiedades de un generador de números aleatorios

Por generador de números pseudoaleatorios entenderemos un algoritmo capaz de producir secuencias de números:

$$u_1, u_2, \dots, u_N,$$

que sean realizaciones de muestras de tamaño N de una variable aleatoria uniforme $U \sim \mathcal{U}(0, 1)$. Para ello debemos acordar qué entendemos por una *muestra* de esta variable.

Definición 3.1. Una N -upla de variables aleatorias $(U_1, U_2, \dots, U_N)$ es una muestra de tamaño N de una variable aleatoria uniforme en $(0, 1)$ si cumple:

1. Para cada $i = 1, \dots, N$ y cada $u \in \mathbb{R}$ vale

$$\mathbb{P}(U_i \leq u) = \begin{cases} 0 & \text{si } u < 0 \\ u & \text{si } 0 \leq u \leq 1 \\ 1 & \text{si } u > 1. \end{cases}$$

2. Para cada k , $2 \leq k \leq N$ y cada k -upla $(i_1, i_2, \dots, i_k)$ con $1 \leq i_1 < \dots < i_k \leq N$ vale que

$$\mathbb{P}(U_{i_1} \leq u_1, \dots, U_{i_k} \leq u_k) = \mathbb{P}(U_{i_1} \leq u_1) \dots \mathbb{P}(U_{i_k} \leq u_k),$$

cualesquiera sean $u_1, \dots, u_k$,

Esta definición significa que no sólo las secuencias de números ($k = 1$) deben estar uniformemente distribuidas en $(0, 1)$, sino que además los pares de números generados (u_{i_1}, u_{i_2}) deben distribuirse uniformemente en un cuadrado de lado 1, las ternas $(u_{i_1}, u_{i_2}, u_{i_3})$ en un cubo de lado 1, y así siguiendo.

Además de satisfacer las propiedades (1) y (2), un buen generador debería satisfacer, en la mejor medida posible, las dos siguientes propiedades:

a) **repetibilidad y portabilidad**, y

b) **velocidad computacional**.

Por **repetibilidad** se entiende que, si en ocasiones repetidas se dan los mismos parámetros, el generador debe producir la misma sucesión. Esta propiedad garantiza que los resultados de una simulación sean confiables.

Portabilidad significa que, sobre las mismas condiciones de definición, la sucesión sea la misma, independientemente del lenguaje computacional usado para implementar el algoritmo de generación, y de la computadora utilizada. Esta propiedad suele ser difícil de alcanzar, pero aún así es un aspecto deseable.

La **velocidad computacional** está estrictamente ligada a la precisión deseada en los resultados finales de la simulación para la cual el generador es utilizado. Cuanto más rápido sea el generador, más resultados serán obtenidos en el mismo tiempo de uso del computador.

De hecho, toda vez que se realiza una simulación, se acepta una solución de compromiso entre los requerimientos expuestos anteriormente, pero siempre los prioritarios debieran ser las propiedades (1) y (2) de la Definición 3.1.

3.2.1. Breve reseña histórica

Los primeros métodos utilizados para obtener secuencias de números aleatorios fueron a través de procedimientos físicos. Existe una larga historia de experimentos de simulación basados en tiradas de monedas o dados, ruletas, que aún hoy son usados en juegos tales como bingos, casinos, loterías, etc. Sin embargo estos procedimientos carecen de la propiedad de repetibilidad.

Probablemente uno de los primeros trabajos publicados sobre generación de números aleatorios fue la tabla de Tippett (1927), que consistía de 10.400 números de cuatro dígitos a partir de 41 600 dígitos aleatorios tomados de datos censales. Lamentablemente no satisfacían ni (1) ni (2) de la Definición 3.1.

La primera máquina generadora de números aleatorios fue usada en 1939 por Kendall y Babington-Smith¹ con la cual produjeron una tabla de 100000 dígitos aleatorios agrupados de a cuatro, formando 25 000 números de cuatro dígitos. También en 1955 la RAND Corporation utilizó extensamente una tabla de 1.000.000 de dígitos aleatorios que fue obtenida a partir de una ruleta electrónica especialmente diseñada. Si bien estos métodos recibieron cierta aceptación, por un lado por satisfacer ciertos tests y/o porque funcionaban en la práctica, tienen la desventaja de no cumplir la condición de repetibilidad y portabilidad. Los métodos para seleccionar una secuencia de números aleatorios no garantizaban la repetibilidad de la secuencia, y la utilización de la tabla requería disponer de ella.

Por lo tanto sólo pueden ser usados en situaciones donde la repetibilidad no es lo buscado, por ejemplo en loterías; o en ciertas aplicaciones estadísticas, como en muestreos aleatorios de poblaciones no muy grandes.

3.3. Principios generales

Al definir un generador, hay ciertos **principios generales** que deben satisfacerse, además de las condiciones que ya hemos marcado:

- P1)** La secuencia generada debe ser intuitivamente aleatoria.
- P2)** Esa aleatoriedad debe ser establecida teóricamente o, al menos, debe pasar ciertos tests de aleatoriedad. La aleatoriedad de una secuencia jamás debe ser asumida sin esas verificaciones.
- P3)** Debe conocerse algo sobre las propiedades teóricas del generador.

¹Tables of Random Sampling Numbers. By M. G. Kendall and B. Babington Smith. Pp. x, 60. 3s. 9d. Tracts for Computers No. xxiv

El principio **P1** es ciertamente razonable. No debería ser posible poder anticipar cuál es el siguiente número en una secuencia si ésta es aleatoria. El segundo principio (**P2**) se refiere a que deben satisfacerse tests estadísticos en relación a las condiciones (1) y (2) de la Definición 3.1. La tercera premisa, (**P3**), es para poder garantizar de manera teórica la condición de repetibilidad del generador.

Estos principios generales indican que los generadores ad hoc deben ser evitados. Es decir, no cualquier algoritmo que genere números *arbitrariamente* entre 0 y 1 debe considerarse un buen generador.

Ejemplo 3.1. Uno de los primeros trabajos que sugieren un método bien definido de generación de una secuencia determinística intentando imitar una secuencia aleatoria, fue de von Neumann. Este método conocido como "mid square" puede ser escrito de la siguiente manera:

1. Sea X_0 un número entero de 4 dígitos decimales (puede ser que los dígitos de más a la izquierda sean 0). Hacer $i = 0$.
2. Calcular X_i^2 . El resultado es un número de 8 dígitos: $X_i^2 = d_8d_7d_6d_5d_4d_3d_2d_1$, con d_j en el conjunto $\{0, 1, \dots, 9\}$, $0 \leq j \leq 7$.
3. Definir $X_{i+1} = d_6d_5d_4d_3$, esto es, los cuatro dígitos centrales o "middle".
4. Hacer $i = i + 1$ y seguir en 2.

```
def vonNeumann(u):
    u= (u**2 // 100) % 10000
    return u
```

Este método es un ejemplo de lo que llamaríamos un **mal generador de números aleatorios**. Veamos por qué.

Aunque ciertas secuencias obtenidas sean bien aleatorias y se repitan sólo después de un número bien grande de términos, sucede que no se conocen bien las propiedades del generador. En particular, pareciera que las secuencias que se obtienen dependen fuertemente del valor inicial X_0 , o semilla. Los siguientes ejemplos muestran algunas secuencias obtenidas a partir de este algoritmo, donde el primer valor es la semilla dada:

- a) 4010 801 6416 1650 7225 2006 240 576 3317 24 5 0 0...
- b) 2100 4100 8100 6100 2100 4100 8100 6100 2100 4100 8100...
- c) 3792 3792 3792 3792 3792 3792...
- d) 1234 5227 3215 3362 3030 1809 2724 4201 6484 422 1780...

En (a) puede verse que la secuencia degenera en el valor 0 luego de algunas iteraciones, en (b) la secuencia alterna entre solo cuatro valores y en (c) los valores se repiten paso tras paso.

3.4. Generadores congruenciales

Es claro que con una computadora no es posible generar cualquier número real, en particular si posee infinitas cifras decimales. Los generadores que analizaremos a continuación producen en realidad una secuencia de números enteros

$$y_1, y_2, \dots, y_N, \quad y_j \in \{0, 1, \dots, M-1\},$$

para un cierto entero positivo M "grande", y a partir de esta muestra se considera la sucesión de números en $[0, 1)$ como

$$u_1 = \frac{y_1}{M}, u_2 = \frac{y_2}{M}, \dots, u_N = \frac{y_N}{M}, \dots$$

Nos ocuparemos entonces de generar secuencias de números enteros:

$$y_n = f(y_{n-1}) \text{ mód } M,$$

para algún M entero positivo, y comenzando de un valor inicial (semilla) y_0 .

3.4.1. Generadores congruenciales lineales

Definición 3.2. Sea M un entero positivo, $M \geq 2$. Una sucesión $y_1, y_2, \dots, y_n, \dots$ con valores en $\{0, 1, \dots, M-1\}$ se dice **generada por el generador congruencial lineal con parámetros a, c y M y semilla y_0** si

$$y_n = (ay_{n-1} + c) \text{ mód } M, \quad n \geq 1,$$

donde a, c e y_0 son enteros del conjunto $\{0, \dots, M-1\}$.

En la terminología usual a se dice un **multiplicador**, c es el **incremento** y M es el **módulo**. Si $c \neq 0$ el generador se dice **mixto** y si $c = 0$ se dice **multiplicativo**.

```
def randMixto(a, c, M, u):
    return (a*u+c) % M

def randMulti(a, M, u):
    return (a*u) % M
```

Ejemplo 3.2. La secuencia

$$0, 1, 6, 15, 12, 13, 2, 11, 8, 9, \dots$$

fue generada por un generador congruencial lineal. El lector, ¿podría decir cual será el próximo número (entre 0 y 15)?

Ejemplo 3.3. La secuencia

$$1, 12, 1, 12, 1, 12, 1, 12, 1, 12, \dots$$

ha sido generada por otro generador congruencial. ¿Podría decirse que genera una secuencia intuitivamente aleatoria?

Los ejemplos anteriores muestran que no siempre una secuencia generada por un generador congruencial lineal resulta intuitivamente aleatoria. En principio pareciera que cuanto más números distintos aparecen en la secuencia hasta obtener un valor repetido, más impredecible será el próximo número. Aunque esto no es del todo cierto, ya que una secuencia del tipo $1, 2, 3, \dots, M-1$, es totalmente predecible.

Una secuencia generada por un generador congruencial lineal puede tener a lo sumo M números diferentes. Además, si un número de la secuencia se repite entonces también se repite la secuencia que sigue a ese número. Entonces, si K es el menor número tal que

$$y_n = y_{n+K}, \quad \text{para todo } n \geq N_0,$$

para algún N_0 , diremos que K es el **período** de la secuencia $y_0, y_1, \dots$.

En principio podemos afirmar que $K \leq M$, es decir, el período no puede superar a la cantidad de números que es posible generar.

Pregunta: Ahora, ¿cómo escoger a , c , M y y_0 para obtener las secuencias con mayor período K posible?

Aún consiguiendo un período K grande, es importante elegir a , c y la semilla y_0 de modo que las secuencias tengan un comportamiento lo más aleatorio posible. Por ejemplo, ¿qué ocurre si elegimos M arbitrario, $a = c = 1$ e $y_0 = 0$? En este caso obtenemos la secuencia

$$0, 1, 2, 3, \dots, M-1,$$

que es de período completo (esto es, $K = M$), pero no es siquiera intuitivamente aleatoria.

Para el caso de generadores congruenciales lineales existen varios resultados teóricos que dan pautas de cómo obtener secuencias con períodos grandes. La aleatoriedad debe ser testeada aparte, esto es, ninguno de estos resultados garantizan que se satisfagan los tests de aleatoriedad.

3.4.2. Generadores mixtos

Algo a notar es que, si el generador es multiplicativo ($c = 0$), entonces un buen generador no debería alcanzar nunca el valor 0, de lo contrario la secuencia degeneraría en una sucesión infinita de ceros. Por lo tanto, para obtener un período máximo, esto es $K = M$, necesariamente debe ser un generador mixto.

Teorema 3.1. Consideremos una secuencia dada por el generador:

$$y_{i+1} = a y_i + c \pmod{M}, \quad c \neq 0.$$

Entonces la secuencia tiene período M si y sólo si se cumplen todas las siguientes condiciones:

- El máximo común divisor entre c y M es 1: $(c, M) = 1$.
- $a \equiv 1 \pmod{p}$, para cualquier factor primo p de M .
- Si 4 divide a M , entonces $a \equiv 1 \pmod{4}$.

La demostración puede encontrarse en Knuth [2].

Damos un ejemplo y sugerimos al lector pensar en otros casos que respondan a este teorema.

Ejemplo 3.4. Consideremos $M = 16$, $c = 3$, $a = 5$. Notemos que si M es una potencia de 2, es relativamente fácil encontrar valores para c y a .

$$y_{n+1} = 5 y_n + 3 \pmod{16}.$$

Entonces:

- Se cumple que $(c, M) = (3, 16) = 1$.
- Se cumple que $5 \equiv 1 \pmod{2}$, donde 2 es el único primo que divide a M .
- 4 divide a 16, y $5 \equiv 1 \pmod{4}$.

Si se cumplen todas las hipótesis, debería obtenerse una secuencia máxima. Tomamos $y_0 = 0$.

$$y_1 = 5 \cdot 0 + 3 \pmod{16}, \quad y_1 = 3,$$

$$y_2 = 5 \cdot 3 + 3 \pmod{16}, \quad y_2 = 2,$$

siguiendo el razonamiento la secuencia continúa:

$$\dots \quad 13 \quad 4 \quad 7 \quad 6 \quad 1 \quad 8 \quad 11 \quad 10 \quad 5 \quad 12 \quad 15 \quad 14 \quad 9 \quad 0 \quad 3 \quad 2 \dots$$

Notemos en particular que, si $M = 2^n$, entonces a debe ser de la forma $4m + 1$ y c debe ser impar. Una ventaja de tomar M igual a una potencia de 2, digamos 2^n , es que computacionalmente tomar módulo equivale a considerar los últimos n bits de la representación.

Ejemplo 3.5. En [5] se presenta el siguiente ejemplo de un modelo de generador provisto por bibliotecas de ANSI C, y que responde a un tipo de generador lineal congruencial mixto. En este caso se toma:

$$a = 1103515245, \quad c = 12345, \quad M = 2^{32}.$$

Así en este caso el período de la secuencia es $K = 2^{32} = 4\,294\,967\,296$.

3.4.3. Generadores multiplicativos

Una desventaja de un generador mixto es que para cada valor generado se efectúa una suma, operación que no se realiza para un generador multiplicativo. Si bien en este caso el período de la secuencia puede ser a lo sumo $M - 1$, es preferible usar este tipo de generadores para ganar en velocidad y costo operacional.

Veamos entonces qué condiciones debe tener un generador multiplicativo para poder obtener un período máximo, $K = M - 1$. Antes de presentar un resultado relacionado con estos generadores, definimos el concepto de **raíz primitiva** de un número natural.

Definición 3.3. Sea M un número natural. Se dice que a es una **raíz primitiva** de M si

$$a^{(M-1)/p} \not\equiv 1 \pmod{M}$$

para cualquier factor primo p de $M - 1$.

Con esta definición, enunciamos el siguiente resultado:

Teorema 3.2. Para un generador multiplicativo

$$y_{i+1} = a y_i \pmod{M},$$

el período K de la secuencia verifica las siguientes tres propiedades:

- Si $K = M - 1$ entonces M es primo.
- Si M es primo, entonces K divide a $M - 1$.
- $K = M - 1$ si y sólo si a es raíz primitiva de M y M es primo.

Entonces el Teorema 3.2 nos da pautas para encontrar generadores multiplicativos con período máximo: determinar un número primo M y una raíz primitiva a de M . Cuanto mayor sea M mayor será el período de la secuencia.

Ejemplo 3.6. Si tomamos $M = 19$, entonces $a = 2$ es una raíz primitiva de M . Para ver esto, notemos que $M - 1 = 18$ es divisible por los primos 2 y 3. Como $a^{18/2} = 2^9$, $a^{18/3} = 2^6$, y 19 no divide a $2^9 - 1$ ni a $2^6 - 1$, entonces 2 es raíz primitiva de $M = 19$.

Si tomamos como semilla $y_0 = 1$ tenemos la secuencia:

1 2 4 8 16 13 7 14 9 18 17 15 11 3 6 12 5 10 1...

que tiene período 19.

Notemos en el Ejemplo 3.6, que los primeros elementos de la secuencia no parecen tan aleatorios. Esto se debe a que la raíz primitiva elegida, 2, es muy pequeña. Luego es conveniente tomar una raíz primitiva más grande para superar más rápidamente el valor de M . Afortunadamente, existen resultados que nos permiten encontrar otras raíces primitivas a partir de una conocida. En particular, si a es raíz primitiva y $(m, M - 1) = 1$, entonces a^m es raíz primitiva.

Entonces, si en el Ejemplo 3.6 tomamos $a = 2^5 = 13 \pmod{19}$, la secuencia obtenida es:

$$1 \quad 13 \quad 17 \quad 12 \quad 4 \quad 14 \quad 11 \quad 10 \quad 16 \quad 18 \quad 6 \quad 2 \quad 7 \quad 15 \quad 5 \quad 8 \quad 9 \quad 3 \quad 1 \quad \dots$$

Ejemplo 3.7. En [5] se menciona el generador congruencial multiplicativo con

$$a = 7^5 = 16807 \quad M = 2^{31} - 1 = 2\,147\,483\,647.$$

En este caso, M es un **primo de Mersenne**, es decir, de la forma $2^k - 1$. La factorización en primos de $M - 1$ está dada por:

$$M - 1 = 2^{31} - 2 = 2 \cdot 3^2 \cdot 7 \cdot 11 \cdot 31 \cdot 151 \cdot 331.$$

Dado que 7 es raíz primitiva de M y $(5, M - 1) = 1$, entonces $7^5 = 16807$ es raíz primitiva, y la secuencia generada tiene período:

$$K = M - 1 = 2\,147\,483\,646.$$

3.4.4. El problema de los hiperplanos

Una de las **desventajas** de cualquier generador congruencial lineal, sea mixto o multiplicativo es que no se cumple tan satisfactoriamente la condición (2) de la Definición 3.1.

Para el caso de los generadores congruenciales lineales, ocurre lo siguiente. Si la secuencia producida es:

$$y_0, y_1, y_2, \dots, y_n, \dots,$$

está demostrado que los puntos $(y_j, y_{j+1}, \dots, y_{j+k-1})$, $j = 0, 1, 2, \dots$ están ubicados en no más de

$$(k!M)^{1/k} = (k!)^{1/k} \sqrt[k]{M}$$

hiperplanos paralelos en $\mathbb{R}^k$. ¿Qué nos dice esto? Que los puntos determinados por estas k -uplas quedan ubicados en una cantidad finita de hiperplanos, y por lo tanto existen "franjas" en las que no cae ninguna k -upla. La Figura 3.1 ilustra esta situación para pares ordenados ($k = 2$) en el caso de algunos generadores congruenciales lineales.

Un mal generador y ejemplo de este problema es **RANDU**. Este es un generador del tipo congruencial lineal multiplicativo ($c = 0$), con $M = 2^{31}$ y $a = 2^{16} + 3$:

$$y_n = 65539 \cdot y_{n-1} \pmod{2\,147\,483\,648}.$$

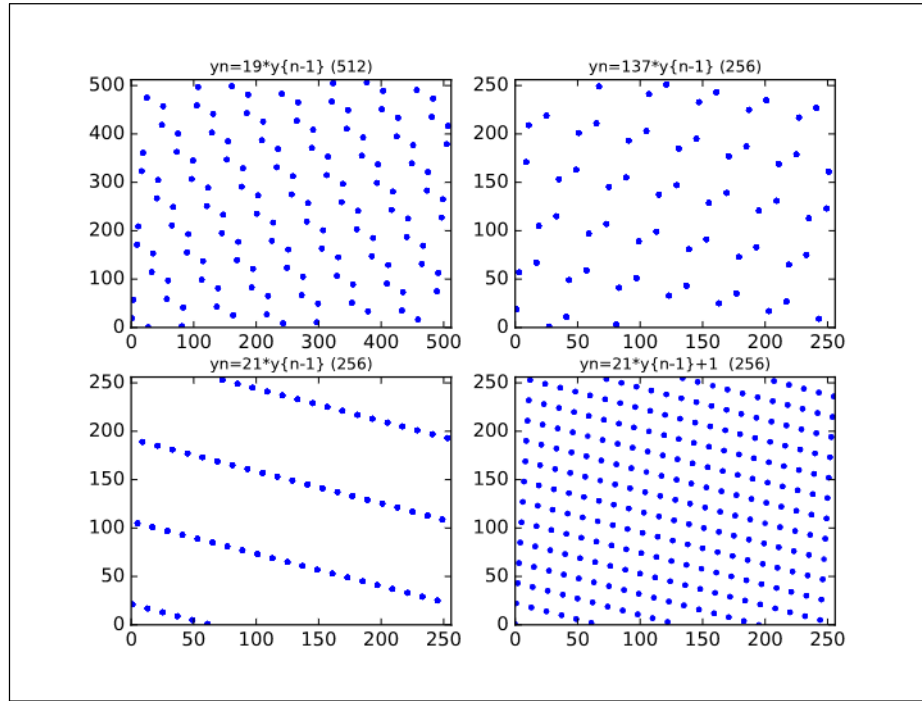


Figura 3.1: Generadores congruenciales. Distribución de pares (y_i, y_{i+1})

Las ternas generadas por RANDU se ubican en 15 planos paralelos, dentro del cubo de lado 1. Figura 3.2. Cabe mencionar que este generador fue implementado en computadoras de IBM por mucho tiempo, y difundido a otros sistemas.

El período de la secuencia para $y_0 = 1234567890$ es $K = 268.435.466$ y para $y_0 = 1$ es 536.870.912.

3.4.5. Generadores congruenciales lineales combinados

Una forma de producir *mayor aleatoriedad* en una secuencia, y evitar la desventaja de los hiperplanos mencionada en la sección anterior, es utilizar más de un generador congruencial lineal y combinarlos entre ellos. Por combinar entendemos sumarlos o restarlos.

Ahora bien, ¿es cierto que si sumamos dos variables aleatorias uniformes obtenemos otra distribución uniforme? La respuesta es no, y podemos verlo arrojando dos dados. Supongamos que sus caras son igualmente probables, entonces las sumas no lo son: el 7 es más probable que el 2, por ejemplo:

$$2 = 1 + 1$$

$$3 = 1 + 2 = 2 + 1$$

$$4 = 1 + 3 = 2 + 2 = 3 + 1$$

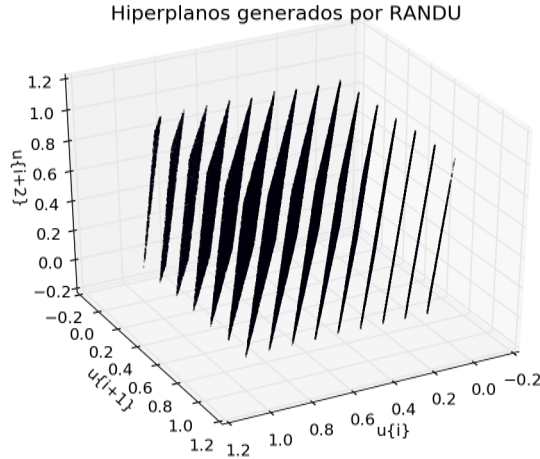


Figura 3.2: Generador RANDU

Sin embargo, si sumamos las caras y consideramos el resultado módulo 6, entonces, vemos que:

$$\begin{aligned}
 1 &= 1 + 6 = 2 + 5 = 3 + 4 = 4 + 3 = 5 + 2 = 6 + 1 && \text{mód } (6) \\
 2 &= 2 + 6 = 3 + 5 = 4 + 4 = 5 + 3 = 6 + 2 = 1 + 1 && \text{mód } (6) \\
 3 &= 3 + 6 = 4 + 5 = 5 + 4 = 6 + 3 = 2 + 1 = 1 + 2 && \text{mód } (6) \\
 4 &= 4 + 6 = 5 + 5 = 1 + 3 = 2 + 2 = 3 + 1 = 6 + 4 && \text{mód } (6) \\
 5 &= 5 + 6 = 6 + 5 = 1 + 4 = 2 + 3 = 3 + 2 = 4 + 1 && \text{mód } (6) \\
 0 &= 6 + 6 = 1 + 5 = 2 + 4 = 3 + 3 = 4 + 2 = 5 + 1 && \text{mód } (6)
 \end{aligned}$$

Esto es, ahora cada suma tiene la misma probabilidad de ocurrir, y por lo tanto **la suma módulo 6** tiene distribución uniforme.

Ejercicio 3.1. ¿Qué pasaría si uno de los dados tuviera 5 caras, y consideráramos las sumas módulo 6?, ¿y si se consideran módulo 5?

Este resultado se generaliza en el siguiente teorema:

Teorema 3.3. Sean $W_1, W_2, \dots, W_n$ variables aleatorias discretas independientes, tales que $W_1 \sim U(\{0, d-1\})$ para cierto $d \geq 1$. Entonces

$$W = \left(\sum_{j=1}^n W_j \right) \text{ mód } d$$

es una variable aleatoria uniforme discreta en $\{0, d-1\}$.

Notemos que sólo se requiere que una de las variables aleatorias sea uniforme en $\{0, d-1\}$. La demostración puede encontrarse en [3] y es la siguiente. Es suficiente considerar $n = 2$, ya que $W_2 + \dots + W_n$ es una variable aleatoria discreta. Entonces, para cada k , $0 \leq k \leq d-1$

$$P(W_1 + W_2 \equiv k \pmod{d}) = P(W_1 + W_2 = k + md, \text{ para algún } m).$$

Pero para cada $m \in \mathbb{Z}$,

$$P(W_1 + W_2 = k + m \cdot d) = \sum_{i=0}^{d-1} P(W_1 = i)P(W_2 = k - i + m \cdot d) = \frac{1}{d} \sum_{i=0}^{d-1} P(W_2 = k - i + m \cdot d).$$

Sumando sobre todos los valores de m posibles, resulta

$$P(W_1 + W_2 \equiv k \pmod{d}) = \frac{1}{d}.$$

Con este resultado vemos que si en particular se suman dos o más generadores congruenciales, tomando módulo uno de los módulos de ellos se obtiene un nuevo generador. ¿Cuál es la ventaja de hacer esto? Recordemos que los generadores congruenciales tienen un determinado período. Si sumamos dos generadores con un mismo período K , entonces K será el período de la suma. Pero si los períodos son diferentes, entonces la suma tiene un período mayor.

Este resultado se basa en el siguiente teorema, y su demostración puede verse en [3].

Teorema 3.4. Consideremos una familia de N generadores, donde para cada j , $j = 1, 2, \dots, N$, el generador j tiene período K_j y evoluciona de acuerdo a una ley:

$$y_{n,j} = f_j(y_{n-1,j}), \quad n \geq 1, \quad y_0 = \text{semilla del generador } j.$$

Entonces el período K de la secuencia

$$s_n = (y_{n,1}, y_{n,2}, \dots, y_{n,N}), \quad n \geq 1,$$

es igual al mínimo común múltiplo de $K_1, K_2, \dots, K_N$.

Así, los Teoremas 3.3 y 3.4 constituyen la base teórica que nos permite obtener nuevos generadores congruenciales de variables aleatorias uniformes a partir de la suma o resta de dos o más generadores. Además, con una buena elección de los períodos de los generadores se podrá garantizar un período mucho más largo para la combinación.

Ejemplo 3.8. Un generador de estas características es el propuesto en [3]. Se consideran los generadores:

$$x_n = 40014x_{n-1} \pmod{2^{31} - 85}$$

$$y_n = 40692y_{n-1} \pmod{2^{31} - 249}$$

Los períodos de estos generadores tienen un solo 2 como factor común:

$$2^{31} - 86 = 2 \cdot 3 \cdot 7 \cdot 631 \cdot 81031, \quad 2^{31} - 250 = 2 \cdot 19 \cdot 31 \cdot 1019 \cdot 1789,$$

por lo que el período de la secuencia $(x_n - y_n) \bmod M$ (para cualquiera de los módulos M) es del orden del producto de los dos períodos dividido 2. En este caso,

$$K \approx 2^{61} = 2\,305\,843\,009\,213\,693\,952 \sim 2.3 \times 10^{18}.$$

La Figura 3.3 ilustra una muestra de aproximadamente 5000 puntos (U_i, U_{i+1}) para cada uno de los tres generadores, con $U_i < 0.001$.

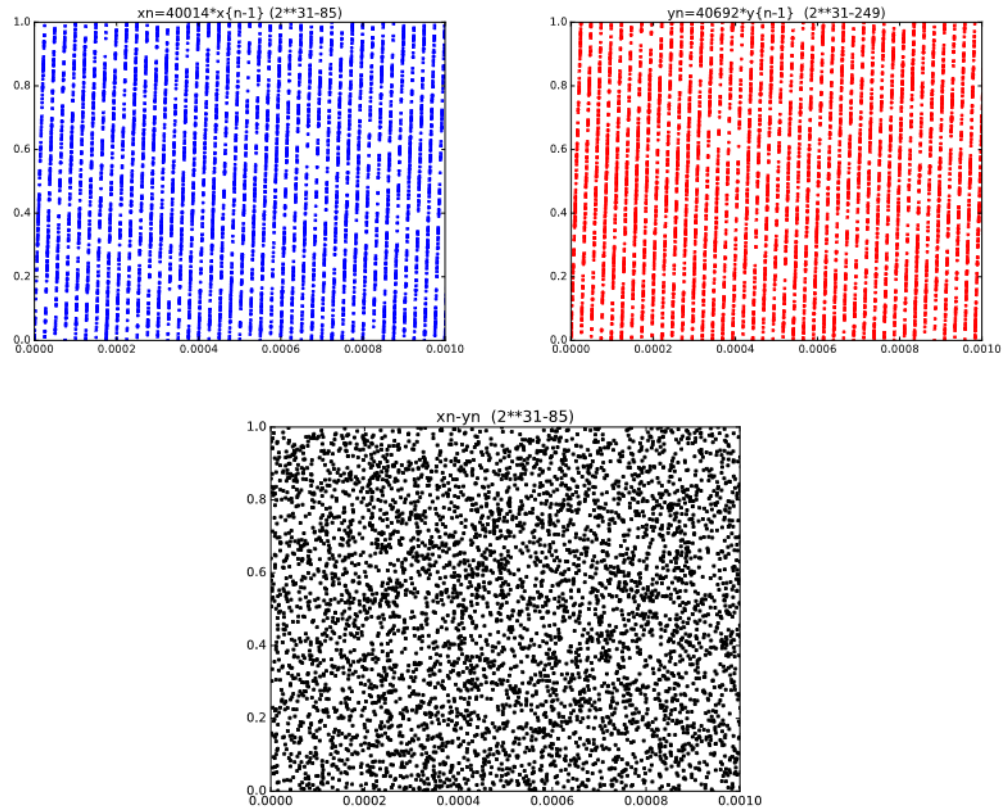


Figura 3.3: Generador del Ejemplo 3.8.

3.4.6. Otros generadores eficientes y portables

Existen otras formas de mejorar la eficiencia de los generadores construidos a partir de un generador congruencial, como son del tipo Fibonacci, resta con préstamo, suma con acarreo, y

otros tantos. En [4] se presenta una extensa lista de generadores candidatos a ser combinados para obtener aún mejores generadores. Algunos son los siguientes:

Módulo	Secuencia	Período
2^{32}	$x_n = 69069 x_{n-1} + \text{impar}$	2^{32}
2^{32}	$x_n = x_{n-1} * x_{n-2}$	2^{31}
2^{32}	$x_n = x_{n-1} + x_{n-2} + C$	2^{58}
$2^{31} - 69$	$x_n = x_{n-3} - x_{n-1}$	2^{62}
$2^{32} - 18$	$x_n = x_{n-2} - x_{n-3} - C$	2^{95}

La constante C indica un 0 o un 1 según corresponda, para acarreos o préstamos.

3.4.7. Mersenne twister

La biblioteca random de Python implementa la rutina **Mersenne-twister**. Este algoritmo es un generador de números pseudoaleatorios desarrollado por Makoto Matsumoto y Takuji Nishimura en 1997. Es ampliamente usado debido a su largo período y buenas propiedades estadísticas:

- Su período es $K = 2^{19937} - 1$
- No presenta correlaciones (formación de hiperplanos).
- Es eficiente computacionalmente.